

Capstone Project Proposal

Submitted to

Faculty of Engineering and Natural Sciences

Department of *Computer Engineering*

In partial fulfillment of the requirements for the degree

BACHELOR of SCIENCE in *COMPUTER ENGINEERING*

“FAgentLLM: Five Agents, One Vision: Smarter Finance, Better Decisions”

Student:

Misem Mohamed

Ecem Oduncu

Anas Brkji

ID Number:

220901646

210901039

220901178

Mentor(s):

Assist. Prof. Dr. Hüsamettim Osmanoglu

Istanbul

January 2026

Abstract

Modern enterprise finance increasingly faces an automation paradox, in which Enterprise Resource Planning (ERP) and Robotic Process Automation (RPA) systems accelerate isolated operational tasks while simultaneously reinforcing fragmented data silos and diffusing decision intelligence across narrowly specialized tools. This proposal introduces FAgentLLM, a unified Multi-Agent System (MAS) architecture that addresses this limitation by positioning Large Language Models (LLMs) as a cognitive orchestration layer for enterprise financial operations. The system integrates five role-specialized agents; Invoice Management, Budget Management, Reconciliation, Credit Tracking and Collection, and Cash Management; within a shared financial state that enables cross-domain causal reasoning, allowing localized actions such as invoice approvals to dynamically inform liquidity forecasts, credit exposure, and collection strategies. Guided by a Design Science Research (DSR) methodology, the project develops and evaluates a functional prototype that bridges probabilistic language-based reasoning with the deterministic execution constraints of regulated financial environments. Key contributions include the implementation of transparent reasoning traces to support Explainable Artificial Intelligence (XAI) and the validation of architectural patterns for compliant deployment of LLM-driven agents, with targeted performance outcomes exceeding 85% field extraction accuracy and 90% reconciliation matching rates, significantly reducing manual processing while preserving auditability and regulatory compliance.

Keywords: Multi-Agent Systems (MAS); Large Language Models (LLMs); Financial Intelligence; Cognitive Orchestration; Explainable AI (XAI)

1. Problem Statement

Modern enterprises have invested heavily in automating financial operations through Enterprise Resource Planning (ERP) systems, Robotic Process Automation (RPA), and specialized software solutions. Despite these investments, finance departments continue to struggle with operational inefficiencies, data fragmentation, and limited strategic visibility. This contradiction reveals a fundamental limitation: current automation accelerates individual tasks but fails to deliver systemic financial intelligence.

1.1 The Automation Paradox in Enterprise Finance

Despite the implementation of ERP and RPA solutions, operational friction and oversight costs continue to rise. This stems from an architectural failure where intelligence is atomized into specialized tools that execute effectively in isolation but fail collectively.

1.1.1 Fragmented Financial Operations

Enterprise financial management spans multiple interdependent domains like invoice processing, budget planning, account reconciliation, credit management, and cash flow monitoring. Each domain typically operates through isolated systems with distinct data formats, user interfaces, and workflow logic. This fragmentation creates several critical problems:

Data Silos and Integration Failures: Financial data remains trapped within departmental boundaries. Invoice data captured in accounts payable systems does not automatically inform budget tracking modules. Reconciliation discrepancies detected in one system fail to trigger risk assessment updates in credit management platforms. Cash flow projections remain disconnected from actual payment behaviors tracked in collection systems.

Manual Coordination Overhead: Finance professionals spend significant time manually reconciling information across systems, validating data consistency, and coordinating actions that should flow naturally from integrated intelligence. A budget manager reviewing expenditures cannot directly access invoice-level details. A credit analyst evaluating customer risk lacks real-time visibility into payment patterns captured during reconciliation.

Loss of Contextual Intelligence: Pre-automation financial operations, while slow and error-prone, possessed inherent contextual awareness. An accountant reviewing an invoice considered vendor history, budget availability, cash position, and strategic priorities simultaneously. Modern automated systems decompose this holistic judgment into discrete validation checks, losing the integrative intelligence that made manual processes effective.

1.1.2 Limitations of Rule-Based Automation

Current financial automation relies predominantly on rule-based logic: predefined workflows, static validation criteria, and template-driven document processing. While effective under stable conditions, these systems exhibit critical weaknesses:

Brittleness to Variability: Rule-based systems fail when confronted with novel document formats, unexpected data structures, or evolving regulatory requirements. Each deviation from programmed expectations requires manual intervention or system reconfiguration.

Lack of Semantic Understanding: Traditional automation extracts data through pattern matching and template recognition but cannot interpret meaning. Systems process invoices without understanding vendor relationships, reconcile transactions without recognizing systematic patterns, and flag exceptions without explaining their financial significance.

Inability to Reason Across Domains: Most critically, existing systems cannot reason about causal relationships between financial domains. An invoice approval system operates independently of liquidity constraints. A budget forecasting model ignores downstream credit exposure. A reconciliation tool treats discrepancies as isolated errors rather than potential indicators of systemic process failures.



Figure 1 Rule-Based Automation Limitations

1.2 The Need for Intelligent Financial Orchestration

As enterprise finance scales, isolated automation tools struggle to maintain coherence, transparency, and control across interconnected processes. Without intelligent orchestration, local optimizations can lead to systemic inefficiencies and increased operational risk, motivating the need for a coordinated, context-aware financial intelligence layer.

1.2.1 From Task Automation to Systemic Intelligence

The fundamental research problem addressed by this project is: How can enterprise financial systems evolve from executing isolated tasks to providing coordinated, context-aware, and explainable decision support across interconnected financial domains?

Solving this problem requires moving beyond incremental improvements in individual automation tools toward a paradigmatic shift in how financial intelligence is architected. The system must:

1. Integrate semantic reasoning to interpret unstructured financial documents and extract meaning beyond pattern recognition
2. Model causal dependencies between financial domains to enable cross-functional coordination
3. Provide explainable outputs that allow finance professionals to understand, audit, and justify automated recommendations
4. Adapt dynamically to evolving formats, regulations, and organizational structures without requiring extensive retraining
5. Maintain governance and compliance by preserving approval hierarchies, validation checkpoints, and audit trails.

1.2.2 The Role of Large Language Models

Recent advances in Large Language Models present a transformative opportunity. LLMs demonstrate unprecedented capabilities in understanding natural language, reasoning over context, and generating human-interpretable explanations. However, their application to enterprise financial systems remains largely unexplored beyond surface-level chatbot interfaces.

The critical research challenge is architecting LLMs not as conversational front ends but as **cognitive orchestration layers** that enhance specialized financial agents with semantic understanding, contextual reasoning, and explainable decision support while preserving the reliability, precision, and governance requirements essential to financial operations.

1.3 Research Significance

This project addresses a gap at the intersection of artificial intelligence, enterprise systems engineering, and financial operations management. The proposed FAgentLLM architecture contributes:

Theoretically: A novel paradigm for LLM-orchestrated multi-agent systems that balances semantic intelligence with structured execution, addressing fundamental limitations in both traditional rule-based automation and purely LLM-based approaches.

Practically: A concrete implementation framework that enterprises can adopt to transform fragmented financial operations into integrated, explainable, and adaptive decision support ecosystems.

Methodologically: Validated design patterns for incorporating LLMs into mission-critical business processes while maintaining accuracy, auditability, and regulatory compliance.

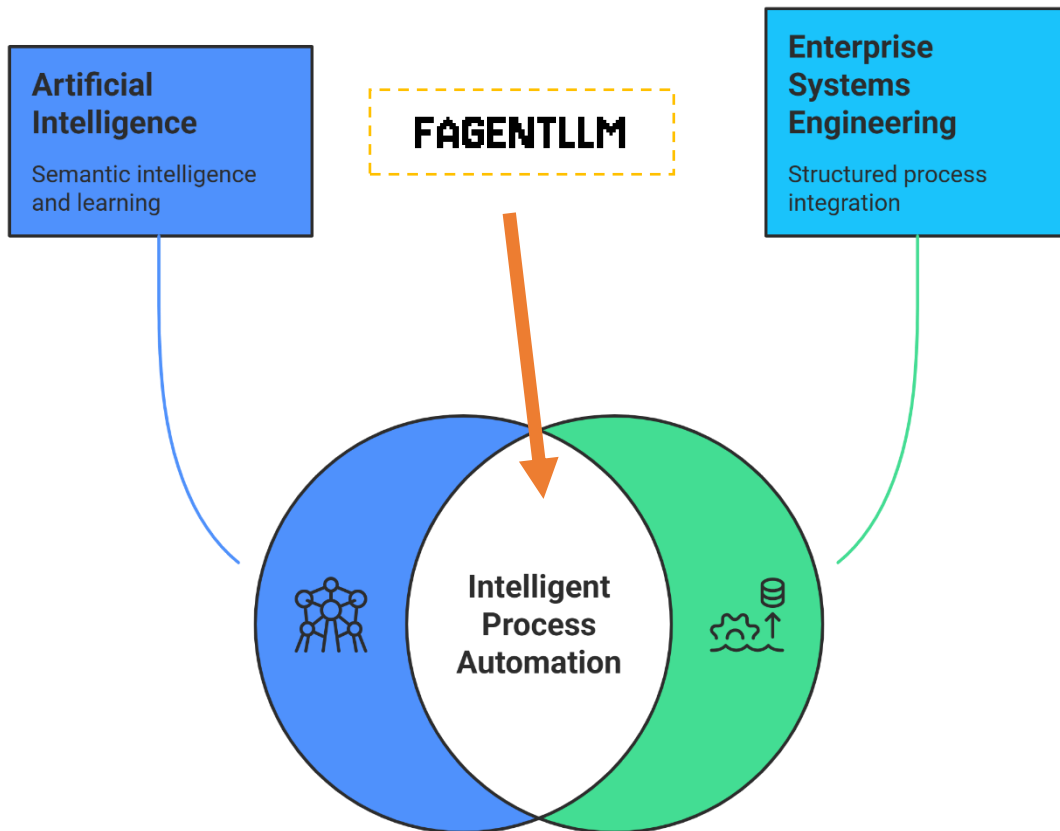


Figure 2 The Intersection of Artificial Intelligence, Enterprise Systems

2. Background

Enterprise financial management has undergone extensive digital transformation over the past two decades through the adoption of ERP systems, financial management platforms, and Robotic Process Automation. These technologies are now deeply embedded in core financial operations such as invoicing, budgeting, reconciliation, credit management, and cash flow monitoring. Despite widespread adoption, their impact remains predominantly operational, motivating a closer examination of how financial automation has evolved and where its limitations persist.

2.1 Evolution of Financial Automation

2.1.1 From Manual Processes to Enterprise Resource Planning

Financial operations historically relied on manual bookkeeping, paper-based approvals, and human judgment. Although labor-intensive and error-prone, these processes possessed inherent contextual awareness as accountants simultaneously considered vendor relationships, budget availability, cash position, and organizational priorities when making decisions.

The introduction of ERP systems in the 1990s promised to digitize and centralize financial operations. Platforms such as SAP, Oracle Financials, and Microsoft Dynamics standardized workflows and consolidated enterprise data into unified schemas. While ERP systems significantly improved consistency and control, they primarily digitized structure rather than judgment. Financial processes were decomposed into rigid entities, relationships, and predefined workflows that function effectively under stable conditions but prove brittle when real-world complexity deviates from encoded assumptions.

2.1.2 Robotic Process Automation and Its Limitations

Robotic Process Automation emerged in the 2010s to automate residual manual tasks not absorbed by ERP systems. RPA tools such as UiPath, Blue Prism, and Automation Anywhere excel at replicating repetitive rule-based actions including data entry, validation, report generation, and system navigation.

However, RPA fundamentally operates as behavioral imitation. Scripts replicate recorded actions with precision but lack semantic understanding. When interfaces change, data formats vary, or unexpected conditions arise, RPA systems fail rather than adapt. As a result, RPA delivers execution without comprehension, speed without intelligence, reinforcing task-level automation rather than enabling systemic reasoning.

2.1.3 The Machine Learning Wave

The integration of machine learning into financial systems introduced genuine analytical capabilities. Techniques such as neural networks and gradient-based models enable invoice field extraction, anomaly detection in transactions, and predictive forecasting. These systems recognize patterns, optimize local objectives, and respond to statistical correlations.

Despite these advances, machine learning approaches remain largely domain-specific. An invoice processing model excels at data extraction but cannot reason about approval logic or liquidity constraints. A forecasting algorithm predicts budget variances without awareness of downstream credit exposure. Performance improvements within isolated domains do not constitute systemic financial intelligence.



Figure 3 Evolution of Financial Automation

2.2 Theoretical Foundations

2.2.1 Large Language Models as Cognitive Infrastructure

Large Language Models, based on the Transformer architecture, employ self-attention mechanisms to model long-range contextual dependencies within sequential data. Through large-scale pre-training on heterogeneous text corpora followed by task-specific fine-tuning, LLMs acquire robust capabilities in semantic understanding, contextual reasoning, and natural language generation.

Within enterprise systems, LLMs are most appropriately conceptualized as an intelligence layer that operates above deterministic computational components. LLMs excel at interpreting unstructured documents, detecting semantic inconsistencies, inferring intent, and generating explainable outputs. However, their probabilistic nature limits reliability for strict arithmetic precision or guaranteed consistency.

This motivates a hybrid architectural paradigm where LLMs provide semantic reasoning and decision guidance while structured systems retain responsibility for validation, execution, and persistence.

2.2.2 Multi-Agent Systems for Distributed Intelligence

Multi-Agent Systems comprise autonomous software entities—agents—that perceive their environment, make decisions, and act to achieve objectives. MAS architectures are characterized by decentralized control, partial observability, and coordinated behavior.

Enterprise financial operations align naturally with this paradigm. Invoice processing, budgeting, reconciliation, credit management, and cash monitoring represent distinct operational responsibilities with different constraints, data sources, and performance criteria. Modeling these as autonomous agents enables modularity, scalability, and fault isolation while preserving coordination capabilities.

2.2.3 LLM-Orchestrated Multi-Agent Architectures

Recent research demonstrates the potential of integrating LLMs as reasoning and orchestration components within agent-based systems. In LLM-orchestrated MAS, language models guide agent behavior by interpreting goals, selecting tools, coordinating actions, and managing inter-agent communication.

This approach differs fundamentally from monolithic LLM systems where a single model attempts all tasks. By embedding LLMs within agents rather than centralizing intelligence, the architecture preserves domain boundaries, enforces governance constraints, and improves explainability. Each agent leverages LLM reasoning to interpret context and propose actions while execution remains constrained by predefined workflows and validation layers.

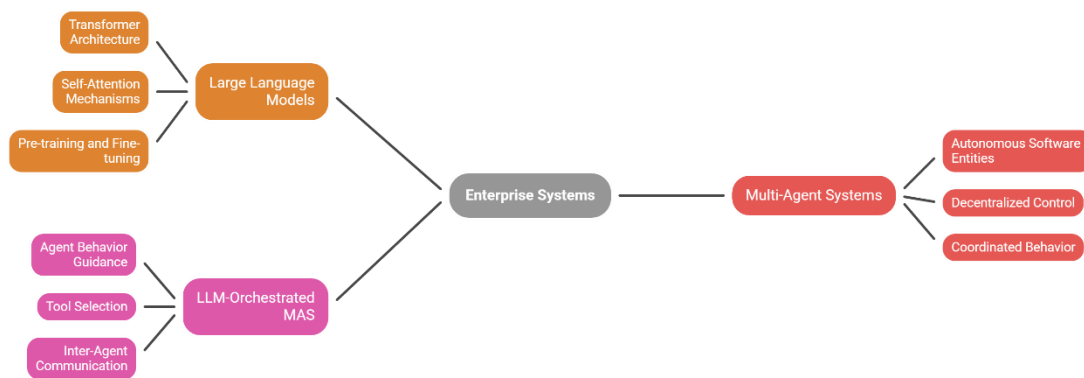


Figure 4 Integrating LLMs and MAS in Enterprise Systems

2.3 Related Work and Research Gap

2.3.1 Domain-Specific Financial Automation

Existing research demonstrates strong performance in isolated financial domains:

Invoice Processing: Loukil et al. (2024) achieve ~90% extraction accuracy using OCR combined with prompt-engineered LLMs. However, their system operates in isolation without approval workflow integration or compliance reasoning.

Budget Management: Huang (2024) integrates temporal convolutional networks with graph neural networks for revenue forecasting achieving ~99.8% accuracy. The model lacks automated alert mechanisms or cross-domain coordination with liquidity systems.

Reconciliation: Bakumenko et al. (2025) pioneer LLM-derived embeddings for heterogeneous financial records, achieving ~95% recall in anomaly detection. The approach relies on synthetic anomalies and lacks explainability for detected discrepancies.

Credit Management: Peng et al. (2025) employ graph-based semi-supervised learning for credit risk assessment, demonstrating strong performance under limited labeled data. The model

focuses on enterprise-level classification rather than invoice-level payment behavior or automated collection workflows.

Cash Flow Forecasting: Tiwari et al. (2025) deliver ~93.4% forecasting accuracy through hybrid LSTM-XGBoost architectures with cloud-based data integration. The framework lacks adaptive learning components and does not reason about cross-domain dependencies.

2.3.2 Integrated Financial Systems

Few studies address multi-agent financial coordination:

Huang et al. (2025) demonstrate edge-deployed LLM agents for invoice reconciliation, achieving 99.9% success rates with role-specialized agents. While validating agent-based architecture, the system focuses narrowly on reconciliation without broader financial ecosystem integration.

Bangun et al. (2025) propose multi-agent conversational systems for stock market analysis, employing fundamental, technical, and macroeconomic reasoning agents. The approach validates structured agent coordination but operates in investment analysis rather than operational finance.

2.3.3 Identified Research Gaps

Current literature reveals five critical gaps:

1. **Absence of Causal Cross-Domain Integration:** No existing system models how financial decisions propagate causally across domains (e.g., invoice approvals affecting liquidity, reconciliation discrepancies informing credit risk).
2. **Limited LLM Integration Beyond Extraction:** LLMs are primarily used for document processing rather than cognitive orchestration, reasoning, and explainable decision support.
3. **Insufficient Explainability:** High-accuracy models provide predictions without interpretable reasoning traces necessary for audit, compliance, and professional judgment.
4. **Static Workflow Assumptions:** Systems rely on predefined rules and historical patterns, lacking adaptability to novel formats, regulatory changes, or organizational evolution.
5. **Lack of Operational Validation:** Research evaluates systems on controlled datasets rather than testing resilience under real-world complexity—format variability, incomplete data, contradictory information.

FAgentLLM directly addresses these gaps by architecting LLMs as a cognitive orchestration layer within a multi-agent financial ecosystem, enabling semantic reasoning, causal coordination, explainable outputs, and adaptive intelligence while maintaining governance and compliance requirements.

3. Project Goals and Objectives

3.1 Primary Goal

The overarching goal of this project is to design, implement, and evaluate FAgentLLM—a unified multi-agent architecture that transforms fragmented enterprise financial operations into an integrated, explainable, and adaptive decision support ecosystem through LLM-powered cognitive orchestration.

3.2 Specific Objectives

3.2.1 Architectural Design Objectives

Objective 1: Design a Modular Multi-Agent Financial Architecture

Develop a comprehensive system architecture comprising five specialized financial agents; Invoice Management, Budget Management, Reconciliation, Credit Tracking and Collection, and Cash Management; operating over a shared financial state representation.

Success Criteria:

- Clear agent responsibility boundaries and interaction protocols
- Defined shared state schema capturing cross-domain financial dependencies
- Event-driven communication mechanisms enabling asynchronous coordination
- Scalable design supporting incremental agent addition or modification

Objective 2: Integrate LLMs as Cognitive Orchestration Infrastructure

Architect Large Language Models as intelligence layers within each agent, enabling semantic document interpretation, contextual reasoning, and explainable decision support while preserving deterministic execution for critical operations.

Success Criteria:

- Hybrid architecture balancing LLM reasoning with structured validation
- Prompt engineering frameworks for domain-specific financial tasks
- Error handling mechanisms addressing LLM probabilistic outputs
- Governance layers ensuring compliance and auditability

3.2.2 Implementation Objectives

Objective 3: Implement the Invoice Management Agent

Develop a functional invoice processing system that automatically extracts data from diverse document formats, validates information against business rules, routes approvals through digital workflows, and integrates with e-invoicing standards.

Success Criteria:

- OCR-based text extraction from PDF and image invoices
- LLM-powered field recognition and validation
- Configurable approval workflow engine
- E-invoice/e-archive integration prototype
- Extraction accuracy $\geq 85\%$ across diverse formats

Objective 4: Implement the Budget Management Agent

Create a centralized budgeting platform that consolidates departmental data, enables real-time budget tracking, supports scenario analysis, and generates automatic alerts for budget deviations.

Success Criteria:

- Multi-department data integration framework
- Real-time budget vs. actual comparison dashboard
- Scenario planning and what-if analysis tools
- Automated threshold-based alert system
- Budget forecast accuracy within $\pm 10\%$ variance

Objective 5: Implement the Reconciliation Agent

Develop an automated reconciliation system that matches internal transactions with external records, detects discrepancies, generates reconciliation reports, and maintains complete audit trails.

Success Criteria:

- Similarity-based transaction matching algorithm
- Anomaly detection for systematic discrepancies
- Automated reconciliation report generation
- Stakeholder collaboration portal for dispute resolution
- Matching accuracy $\geq 90\%$ with $< 5\%$ false positive rate

Objective 6: Implement the Credit Tracking and Collection Agent

Build a credit management system that monitors payment due dates, performs risk analysis, automates collection workflows, and tracks performance metrics.

Success Criteria:

- Automated due date monitoring and reminder system
- Interpretable credit risk scoring model
- Multi-stage collection workflow automation
- Performance dashboard showing collection efficiency
- Risk prediction accuracy $\geq 80\%$ with explainable outputs

Objective 7: Implement the Cash Management Agent

Construct a cash visibility and forecasting platform that consolidates multi-account balances, projects cash flow, and supports liquidity planning decisions.

Success Criteria:

- Real-time multi-account balance consolidation
- Cash flow forecasting with ≥ 7 -day projection horizon
- Liquidity risk identification and alerting
- Integration with budget and payment data
- Forecast accuracy within $\pm 15\%$ for 7-day horizon

3.2.3 Integration and Evaluation Objectives**Objective 8: Establish Inter-Agent Coordination Mechanisms**

Implement communication protocols and shared state management enabling agents to coordinate decisions based on cross-domain financial dependencies.

Success Criteria:

- Event-driven messaging system for agent communication
- Shared financial state database with transactional consistency
- Coordination scenarios demonstrating causal propagation (e.g., invoice approval considering cash availability)
- Successful execution of ≥ 3 multi-agent coordination workflows

Objective 9: Develop Explainable AI Capabilities

Integrate natural language explanation generation enabling finance professionals to understand, audit, and justify automated recommendations.

Success Criteria:

- LLM-generated reasoning traces for agent decisions
- User-friendly explanation interfaces

- Audit trail documentation meeting compliance standards
- User comprehension validation through feedback evaluation

Objective 10: Evaluate System Performance and Impact

Conduct comprehensive evaluation measuring technical performance, decision support quality, and operational impact compared to baseline manual and traditional automated approaches.

Success Criteria:

- Quantitative performance metrics (accuracy, processing time, error rates)
- Qualitative assessment of explainability and usability
- Comparative analysis vs. baseline approaches
- Documentation of limitations and improvement opportunities



Figure 5 FAgentLLM Development Process

3.3 Deliverables

1. **System Architecture Documentation:** Complete technical specification of FAgentLLM architecture including agent designs, communication protocols, and integration frameworks.
2. **Functional Prototype System:** Working implementation of all five financial agents with basic coordination capabilities.
3. **Performance Evaluation Report:** Comprehensive analysis of system performance including quantitative metrics, comparative benchmarks, and qualitative assessments.
4. **Research Publication Draft:** Academic paper documenting architecture, implementation, and findings suitable for conference or journal submission.
5. **Deployment Guidelines:** Practical documentation supporting enterprise adoption including installation procedures, configuration guides, and best practices.

4. Methods

4.1 Research Methodology Framework

This project employs a Design Science Research (DSR) methodology, which is appropriate for developing and evaluating innovative artifacts that address identified practical problems. The DSR framework comprises six activities: problem identification, objective definition, design and development, demonstration, evaluation, and communication. Our research iterates through these activities across multiple development cycles.

4.1.1 Overall Research Process

Phase 0: Problem Identification and Motivation

- Analyze the current limitations of enterprise financial document processing systems
- Identify inefficiencies inherent in traditional automation approaches
- Document stakeholder pain points and define core business requirements
- Establish the research gap through a structured review of existing literature

Phase 1: Requirements Analysis and Architecture Design

- Conduct detailed analysis of enterprise financial workflows
- Define functional and non-functional requirements for each agent
- Design overall system architecture and agent interaction protocols
- Select appropriate technologies and frameworks

Phase 2: Prototype Implementation

- Develop individual agent prototypes following iterative development cycles
- Implement core LLM integration and document processing capabilities
- Build agent coordination mechanisms and shared state management
- Create user interfaces for agent interaction and monitoring

Phase 3: System Integration and Testing

- Integrate individual agents into unified FAgentLLM system
- Conduct functional testing of end-to-end workflows
- Validate inter-agent coordination scenarios
- Perform security and compliance testing

Phase 4: Evaluation and Refinement

- Execute comprehensive performance evaluation
- Collect quantitative metrics and qualitative feedback

- Conduct comparative analysis against baseline approaches
- Refine system based on evaluation findings

Phase 5: Documentation and Dissemination

- Prepare final project report and academic publication
- Create deployment documentation and user guides
- Develop demonstration materials and presentation
- Plan knowledge transfer and future development roadmap

4.2 System Architecture and Design

4.2.1 Overall Architecture Pattern

FAgentLLM employs a **microservices-inspired architecture** where each financial agent operates as a semi-autonomous service with clear interfaces and responsibilities. The architecture comprises three primary layers:

Intelligence Layer (LLM Core):

- Large Language Model integration for semantic understanding and reasoning
- Prompt engineering framework for domain-specific financial tasks
- Natural language process explanation and generation
- Context management and memory systems

Agent Layer (Business Logic):

- Five specialized financial agents with defined responsibilities
- Agent-specific workflow engines and business rule processors
- Tool integration for external system connectivity
- Local state management and decision logging

Data Layer (Persistence and Integration):

- Shared financial state database capturing cross-domain entities
- Event streaming infrastructure for inter-agent communication
- External system integrations (ERP, banking, e-invoice platforms)
- Audit trail and compliance logging
- We may add tenders community (depending on the dataset)

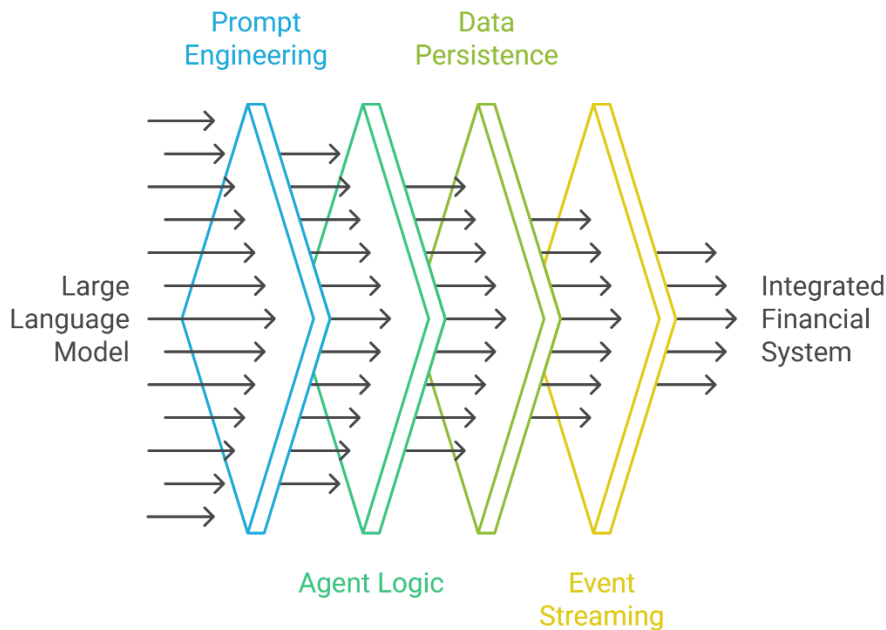


Figure 6 FAgentLLM System Architecture Funnel

4.2.2 Agent Design Pattern

Each agent follows a consistent architectural pattern:

Perception Module: Monitors relevant data sources and events (documents, transactions, schedules, system notifications)

LLM Reasoning Module: Interprets inputs using Large Language Models to extract meaning, assess context, and formulate response options

Decision Module: Evaluates LLM recommendations against business rules, constraints, and validation logic to select appropriate actions

Execution Module: Performs selected actions through deterministic processes (database updates, API calls, workflow triggers)

Communication Module: Broadcasts relevant events to other agents and responds to coordination requests

Explanation Module: Generates natural language justifications for decisions enabling transparency and auditability



Figure 7 Agent Design Pattern

4.3 Technology Stack and Implementation Approach

4.3.1 Core Technologies

Large Language Model Infrastructure:

- Primary LLM: OpenAI GPT-4 or Anthropic Claude for advanced reasoning tasks
- Secondary LLM: Open-source alternatives (Llama 3, Mistral) for cost-sensitive operations
- LLM Framework: LangChain for agent orchestration and tool integration
- Prompt Management: Custom prompt template library with version control

Backend Development:

- Programming Language: Python 3.11+ for primary development
- Web Framework: FastAPI for RESTful API development
- Agent Framework: LangGraph or CrewAI for multi-agent coordination
- Task Queue: Celery with Redis for asynchronous processing

Data Management:

- Primary Database: PostgreSQL for structured financial data
- Vector Database: Chroma or Pinecone for document embeddings
- Event Store: RabbitMQ for inter-agent messaging
- Caching: Redis for performance optimization

Document Processing:

- OCR Engine: Tesseract OCR for text extraction
- PDF Processing: PyMuPDF or pdfplumber for document manipulation
- Layout Analysis: LayoutLM or Donut for structured extraction

Frontend Development:

- Framework: React.js with TypeScript
- UI Library: Material-UI or Ant Design for professional interfaces
- Visualization: Chart.js or D3.js for financial dashboards
- State Management: Redux or Zustand

Infrastructure and Deployment:

- Containerization: Docker for application packaging
- Orchestration: Docker Compose for local development, Kubernetes-ready design
- Cloud Services: AWS or Azure for deployment (S3/Blob Storage, Lambda/Functions)
- CI/CD: GitHub Actions for automated testing and deployment

4.3 Development Methodology

The development of FAgentLLM follows Agile software development principles, selected to support iterative refinement, continuous feedback, and incremental validation of complex multi-agent behavior. Agile methodology is particularly suitable for this project due to the exploratory nature of integrating Large Language Models with enterprise financial workflows.

Development is organized into two-week sprint cycles, each producing a demonstrable and testable increment of system functionality.

Sprint Activities:

- Sprint Planning: Definition of sprint objectives, prioritization of user stories, and task allocation across agents.
- Development: Iterative implementation of agent functionality with continuous integration and testing.

- Sprint Review: Demonstration of completed features and collection of stakeholder feedback.
- Sprint Retrospective: Reflection on technical and process improvements for subsequent sprints.

Code Quality and Engineering Practices:

- Version control using Git with feature-based branching
- Mandatory code reviews for all pull requests
- Automated unit testing with a target coverage exceeding 80%
- Integration testing focused on inter-agent coordination
- Comprehensive documentation using docstrings and README files

4.4 Detailed Implementation Plan for Each Agent

4.4.1 Invoice Management Agent Implementation

Input Processing Pipeline:

1. Document Ingestion: Accept invoices via email parsing, file upload, API integration, or e-invoice systems
2. Format Detection: Identify document type (PDF, image, XML) and vendor format
3. OCR Extraction: Apply Tesseract OCR or cloud-based services for text extraction
4. Layout Analysis: Use layout-aware models to identify invoice structure
5. Field Extraction: Apply LLM with specialized prompts to extract structured fields (vendor, date, amount, line items, tax)

Validation and Enrichment:

1. Syntax Validation: Verify data types, formats, and mandatory field completion
2. Business Rule Validation: Check against vendor master data, purchase orders, price agreements
3. Cross-Reference Validation: Verify budget availability (Budget Agent), check payment capacity (Cash Agent)
4. Anomaly Detection: Flag unusual amounts, duplicate invoices, or suspicious patterns

Approval Workflow:

1. Workflow Routing: Determine approval hierarchy based on amount thresholds, department, vendor category

2. Notification Generation: Send approval requests via email or system notifications
3. Status Tracking: Monitor approval progress, escalate delays
4. Audit Logging: Record all decisions, approvals, and modifications

Integration Points:

- E-invoice platforms for electronic invoice reception
- ERP systems for vendor master data and general ledger posting
- Budget Agent for expenditure authorization
- Cash Agent for payment scheduling
- Reconciliation Agent for payment confirmation

Performance Targets:

- Invoice processing time: <60 seconds per invoice
- Field extraction accuracy: $\geq 85\%$
- Approval routing accuracy: 100%
- E-invoice integration success rate: $\geq 95\%$

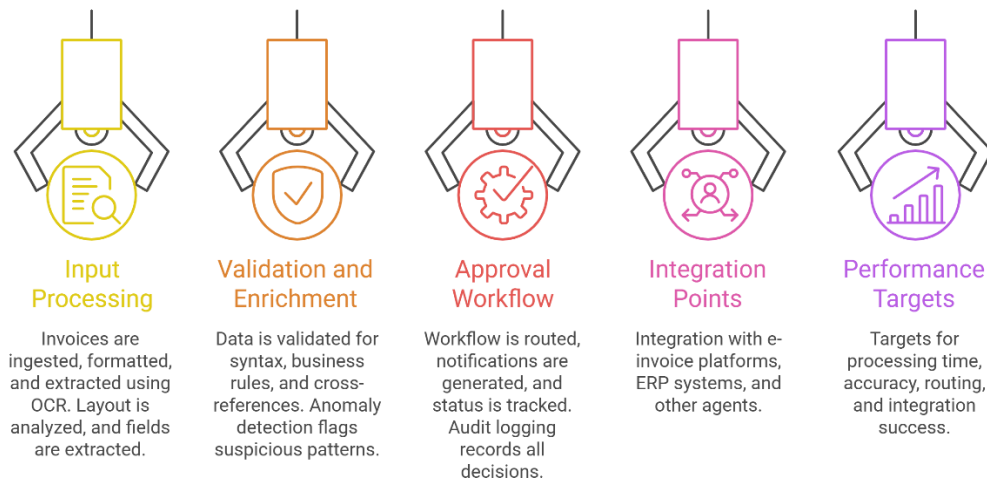


Figure 8 Invoice Management Agent

4.4.2 Budget Management Agent Implementation

Data Integration:

1. Budget Data Collection: Import historical budgets, actuals from ERP, departmental submissions
2. Granularity Mapping: Establish hierarchical budget structure (organization → department → cost center → account)
3. Temporal Alignment: Synchronize multi-period budgets (monthly, quarterly, annual)
4. Data Validation: Ensure consistency, completeness, and reconciliation with general ledger

Budget Planning and Forecasting:

1. Baseline Generation: Calculate baseline budgets from historical spending patterns
2. Scenario Modeling: Enable what-if analysis with adjustable parameters (growth rates, cost changes, strategic initiatives)
3. Forecast Generation: Apply time series forecasting (LSTM, XGBoost) for expenditure predictions
4. LLM-Assisted Planning: Use LLM to interpret strategic goals and recommend budget allocations

Real-Time Monitoring:

1. Actual vs. Budget Comparison: Continuously compare committed and actual spending against budgets
2. Variance Analysis: Calculate variances, identify trends, flag anomalies
3. Alert Generation: Trigger notifications when thresholds exceeded (e.g., >90% budget utilization)
4. Dashboard Updates: Refresh real-time visualizations showing budget health

Integration Points:

- Invoice Agent for expenditure capture
- ERP for actual spending data
- Credit Agent for revenue forecasting
- Cash Agent for liquidity-informed budget adjustments

Performance Targets:

- Forecast accuracy: $\pm 10\%$ variance for monthly predictions

- Alert latency: <5 minutes from threshold breach
- Scenario generation time: <30 seconds
- Dashboard refresh rate: Real-time (event-driven)

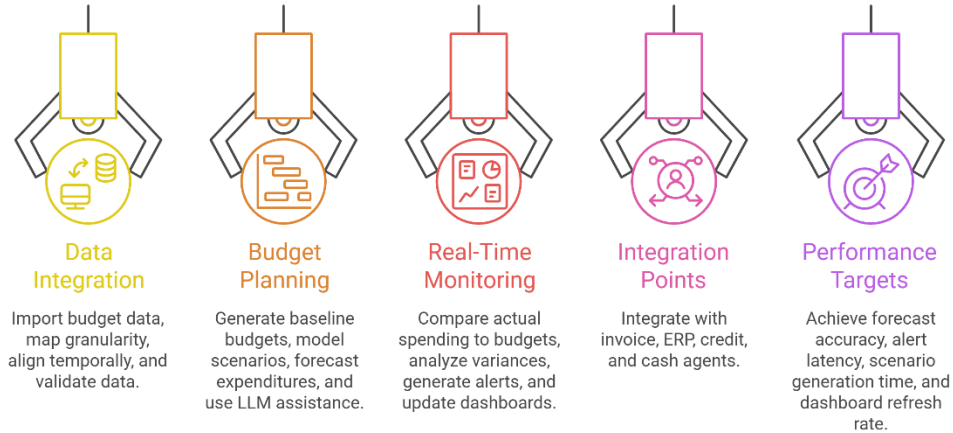


Figure 9 Budget Management Agent

4.4.3 Reconciliation Agent Implementation

Transaction Matching:

1. **Data Ingestion:** Import internal transactions (general ledger, sub-ledgers) and external records (bank statements, supplier statements)
2. **Data Normalization:** Standardize formats, currencies, date representations
3. **Feature Engineering:** Extract matching features (amount, date, counterparty identifiers, descriptions)
4. **Embedding Generation:** Use LLM embeddings (SBERT) to encode transaction descriptions
5. **Similarity Calculation:** Compute cosine similarity between internal-external transaction pairs
6. **Match Identification:** Apply threshold-based or ML classification to determine matches

Discrepancy Detection and Investigation:

1. **Unmatched Transaction Identification:** Flag internal or external records without matches
2. **Anomaly Classification:** Categorize discrepancies (timing differences, amount variances, missing entries, duplicates)
3. **Pattern Recognition:** Use LLM to detect systematic issues (recurring vendor discrepancies, process failures)
4. **Root Cause Analysis:** Generate hypotheses for discrepancies using contextual reasoning

Reconciliation Reporting:

1. Report Generation: Create structured reconciliation reports with matched transactions, discrepancies, and statistics
2. Stakeholder Portal: Provide collaborative interface for resolving disputes with external parties
3. Audit Trail: Maintain complete history of reconciliation activities, decisions, adjustments
4. Analytics Dashboard: Visualize reconciliation metrics, trends, efficiency improvements

Integration Points:

- Bank integration APIs for automated statement import
- Invoice Agent for payment confirmation
- Credit Agent for receivables reconciliation
- Cash Agent for balance validation

Performance Targets:

- Matching accuracy: $\geq 90\%$
- False positive rate: $< 5\%$
- Reconciliation report generation: < 5 minutes for 10,000 transactions
- Discrepancy investigation time reduction: $> 50\%$ vs. manual

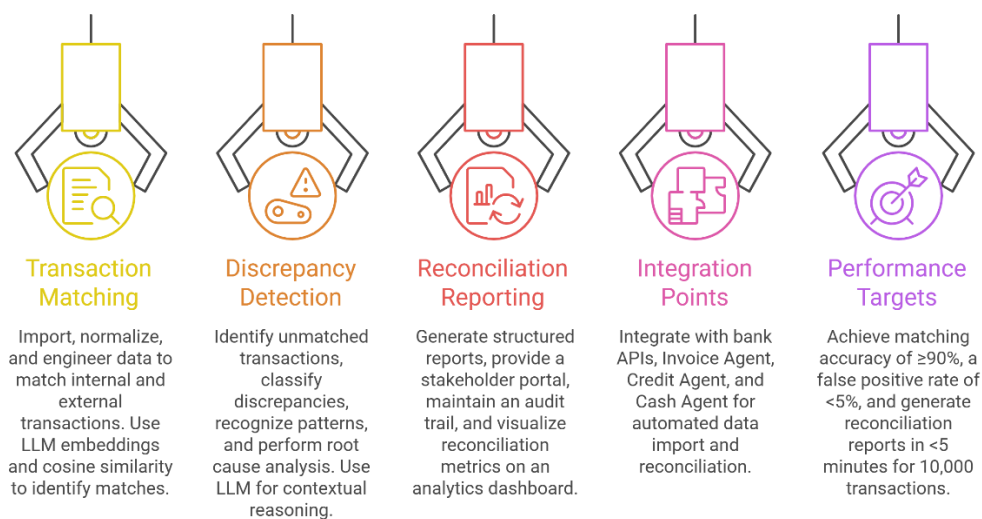


Figure 10 Reconciliation Agent Implementation

4.4.4 Credit Tracking and Collection Agent Implementation

Credit Monitoring:

1. Receivables Tracking: Monitor all outstanding invoices, payment terms, due dates
2. Aging Analysis: Calculate aging buckets (current, 30 days, 60 days, 90+ days)
3. Customer Profile Management: Maintain customer payment history, credit limits, risk scores
4. Due Date Alerts: Trigger proactive reminders before payment due dates

Risk Assessment:

1. Feature Extraction: Collect payment history, dispute frequency, financial indicators, external credit data
2. Risk Scoring Model: Apply interpretable scoring formula combining weighted factors
3. LLM Risk Analysis: Use LLM to interpret qualitative signals (customer communications, market news)
4. Dynamic Score Updates: Refresh risk assessments based on real-time payment behaviors and reconciliation findings

Collection Workflow Automation:

1. Reminder Generation: Create personalized payment reminders using LLM-generated natural language
2. Multi-Channel Distribution: Send reminders via email, SMS, customer portal
3. Escalation Logic: Progress through collection stages (friendly reminder → formal notice → manager escalation → legal referral)
4. Performance Tracking: Monitor collection efficiency, days sales outstanding (DSO), recovery rates

Integration Points:

- Invoice Agent for receivable creation
- Reconciliation Agent for payment confirmation
- Cash Agent for cash flow impact assessment
- External credit bureaus for customer credit information

Performance Targets:

- Risk prediction accuracy: $\geq 80\%$

- DSO reduction: $\geq 15\%$ improvement
- Automation rate: $\geq 70\%$ of collections handled without manual intervention
- Customer satisfaction: No significant complaints about collection approach

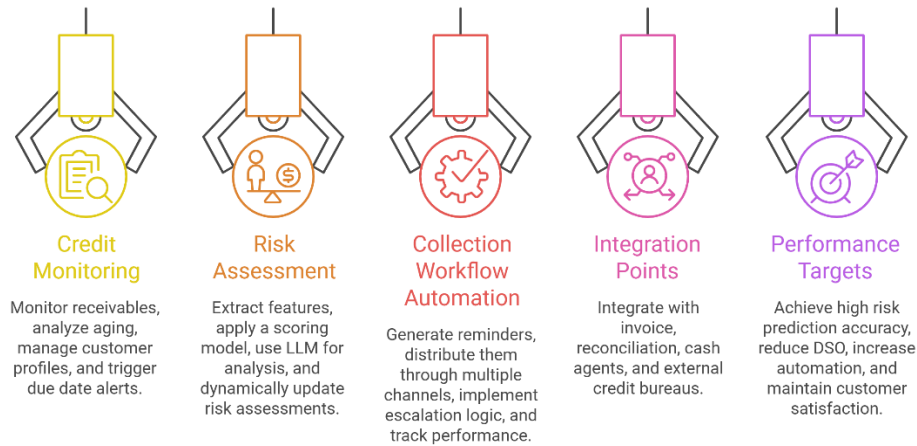


Figure 11 Credit Tracking and Collection Agent

4.4.5 Cash Management Agent Implementation

Cash Position Consolidation:

1. Multi-Account Integration: Connect to multiple bank accounts via APIs (Open Banking, bank-specific APIs)
2. Balance Aggregation: Consolidate real-time balances across accounts, currencies
3. Currency Conversion: Apply current exchange rates for multi-currency portfolios
4. Historical Data Collection: Maintain time series of cash balances, transactions

Cash Flow Forecasting:

1. Inflow Projection: Forecast expected receipts from invoices (Credit Agent), revenue streams
2. Outflow Projection: Predict payments from approved invoices (Invoice Agent), payroll, recurring expenses
3. Time Series Modeling: Apply LSTM or XGBoost for forecasting with ≥ 7 -day horizon
4. LLM-Enhanced Forecasting: Use LLM to incorporate qualitative factors (market conditions, strategic decisions)

Liquidity Analysis and Decision Support:

1. Liquidity Threshold Monitoring: Compare projected cash against minimum operating balances

2. Shortage/Surplus Identification: Flag periods of insufficient or excess liquidity
3. Scenario Analysis: Model cash impacts of alternative decisions (payment timing, investment options)
4. Recommendation Generation: Provide decision support for liquidity management, investment, financing

Integration Points:

- Invoice Agent for payment obligations
- Budget Agent for planned expenditures
- Credit Agent for expected receivables
- Banking systems for account balances and transactions

Performance Targets:

- Forecast accuracy: $\pm 15\%$ for 7-day horizon
- Balance consolidation latency: < 10 seconds
- Liquidity alert generation: < 5 minutes from threshold breach
- Scenario analysis response time: < 30 seconds

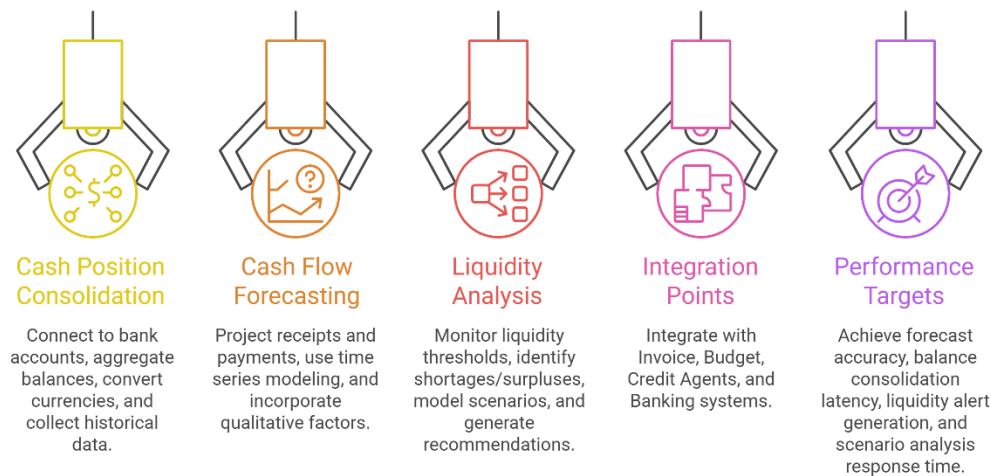


Figure 12 Cash Management Agent

4.5 LLM Integration Strategy

4.5.1 Prompt Engineering Framework

Prompt engineering serves as a foundational control mechanism for achieving reliable, task-specific LLM behavior. Each agent employs a library of domain-specific prompt templates that explicitly define the agent's role, the expected input context, and the required output structure. Prompts are intentionally constrained and schema-driven to minimize ambiguity, reduce

hallucination risk, and support downstream validation.

Domain-Specific Prompt Templates

Each financial agent maintains reusable prompt templates aligned with its functional responsibilities, including document understanding, risk assessment, and exception explanation.

Illustrative Invoice Extraction Prompt

You are a financial document processing specialist.

Extract structured information from the following invoice text.

Invoice Text:

{ocr_output}

Extract and return a JSON object with the following fields:

- vendor_name
- vendor_tax_id
- invoice_number
- invoice_date
- due_date
- total_amount
- currency
- line_items (array of {description, quantity, unit_price, total})
- tax_amount

If any field cannot be determined, we can use null.

Ensure all monetary values are numeric.

Illustrative Invoice Extraction Prompt



Figure 13 Illustrative prompt template used by the Invoice Management Agent for structured information extraction from unstructured invoice documents.

Illustrative Credit Risk Analysis Prompt

You are a credit risk analyst.

Analyze the following customer information and provide a risk assessment.

Customer Profile:

{customer_data}

Payment History:

{payment_history}

Current Outstanding Balance:

{outstanding_balance}

Provide:

- Risk level (Low / Medium / High)
- Key risk factors (bullet list)
- Recommended actions
- Explanation (2–3 sentences)

Be concise and focus on actionable insights.

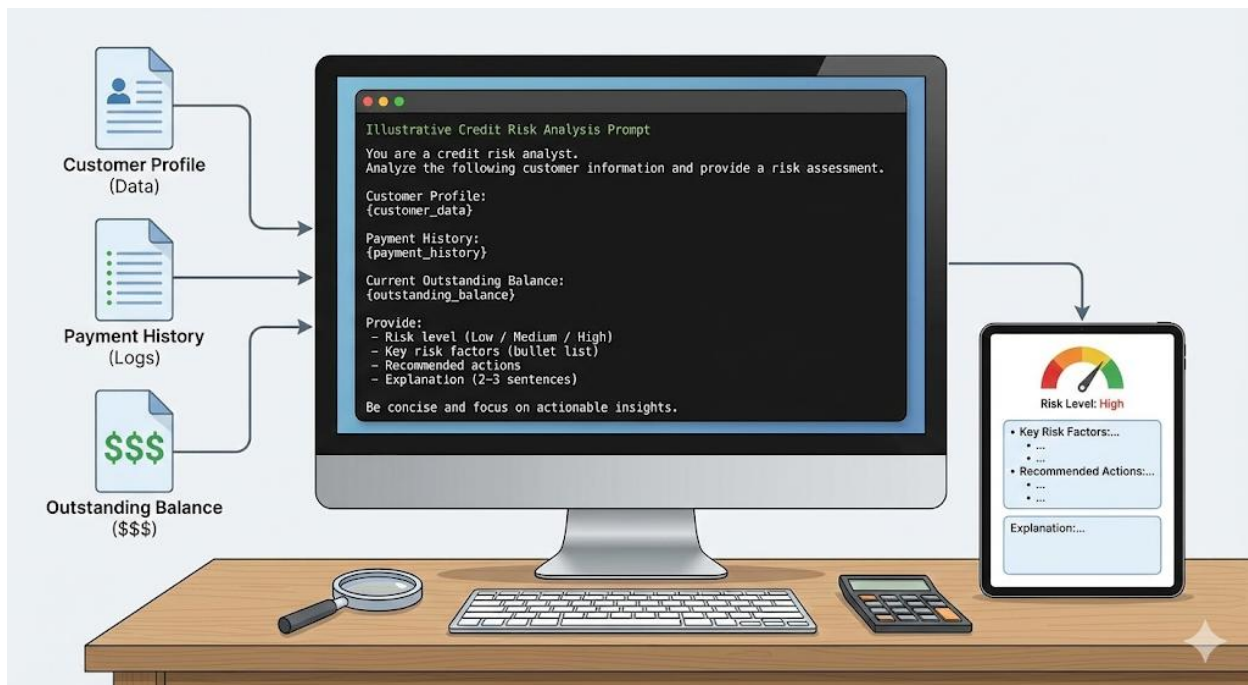


Figure 14 Illustrative prompt-driven credit risk analysis workflow employed by the Credit Tracking Agent, integrating customer profiles, payment history, and outstanding balances to generate explainable risk assessments.

These prompt templates ensure consistency across executions, enforce structured outputs, and enable systematic validation against predefined schemas and business rules.

4.5.2 LLM Selection and Optimization

Model selection within FAgentLLM is driven by task requirements and operational constraints rather than reliance on a single universal model.

Model Selection Criteria

- **Task Complexity:** Lightweight extraction tasks employ smaller or open-source models, while complex reasoning and cross-domain analysis leverage high-capability models (e.g., GPT-4-class or Claude-class models).
- **Latency Requirements:** Real-time decision support prioritizes low-latency models, cached embeddings, and asynchronous execution.
- **Cost Constraints:** High-volume or repetitive tasks favor cost-efficient or self-hosted models.
- **Accuracy and Criticality:** High-impact financial decisions utilize the most accurate models, supplemented with additional validation layers.

Performance Optimization Techniques

- Prompt caching to reduce repeated token usage
- Batch processing of semantically similar requests

- Fallback model chains (primary → secondary model)
- Schema-based response validation
- Optional fine-tuning on domain-specific financial data to improve extraction accuracy and reasoning consistency

4.5.3 LLM Reliability and Governance

Given the mission-critical nature of financial systems, LLM outputs are never accepted without verification. Reliability is enforced through layered safeguards that combine probabilistic intelligence with deterministic controls.

Error Handling and Control

- Validation of output schemas, data types, and mandatory fields
- Consistency checks against business rules and historical data
- Confidence-based human-in-the-loop escalation for uncertain outputs
- Deterministic rule-based fallbacks when LLM outputs are invalid or unavailable

Compliance and Explainability

- Comprehensive logging of LLM inputs, outputs, and execution context
- End-to-end audit trails linking source documents to final decisions
- Generation of user-facing explanations to justify automated recommendations
- Alignment with enterprise governance, auditing, and reporting standards

4.6 Data Collection and Preparation

4.6.1 Training and Test Data

Multiple datasets are used to evaluate the system across diverse financial scenarios and operational conditions.

Invoice Dataset

- Sources: Public datasets (e.g., SROIE, CORD) supplemented with synthetically generated invoices
- Size: Approximately 5,000 invoices
- Coverage: Diverse layouts, languages, currencies, and vendor formats
- Annotation: Ground truth labels for vendor details, dates, amounts, taxes, and line items
- Augmentation: OCR noise, layout distortions, and formatting variations

Transaction Dataset

- Source: Synthetic generation using realistic financial distributions

- Size: Approximately 100,000 transactions
- Characteristics: Matched pairs, partial matches, discrepancies, and anomalies
- Validation: Manual inspection of representative samples

Budget and Cash Flow Data

- Sources: Public financial reports, anonymized enterprise data (when available), and synthetic datasets
- Structure: Multi-year time series across 50+ departments or cost centers
- Features: Budgets, actuals, forecasts, and variance indicators

Credit and Collection Data

- Source: Synthetic customer profiles
- Size: Approximately 1,000 accounts with two-year payment histories
- Distribution: Balanced representation across low-, medium-, and high-risk categories

4.6.2 Data Quality and Ethics

Data Validation

- Completeness and consistency checks
- Duplicate detection and correction
- Standardization of formats, currencies, and units

Privacy and Security

- Full anonymization of personally identifiable information
- Data masking for sensitive financial attributes
- Secure storage with encryption
- Compliance with GDPR and financial data protection regulations

Bias Mitigation

- Diverse representation across document types and organizational units
- Fairness testing in credit risk predictions
- Continuous monitoring for unintended discriminatory patterns

4.7 Evaluation Methodology

4.7.1 Performance Metrics

Agent-Specific Metrics

Invoice Management Agent

- Field extraction accuracy (precision, recall, F1-score)
- Average processing time per invoice
- Approval routing accuracy
- E-invoice integration success rate

Budget Management Agent

- Forecast accuracy (MAPE, RMSE)
- Alert precision and recall
- Scenario generation latency
- User-perceived decision support value

Reconciliation Agent

- Transaction matching accuracy
- False positive and false negative rates
- Reconciliation cycle time
- Discrepancy resolution success rate

Credit Tracking Agent

- Risk prediction accuracy (AUC-ROC)
- Days Sales Outstanding (DSO) improvement
- Collection automation rate
- Early delinquency detection rate

Cash Management Agent

- Cash flow forecast accuracy (MAE, MAPE)
- Liquidity alert timeliness
- Balance consolidation accuracy
- Scenario analysis quality

System-Level Metrics

- End-to-end process completion time
- Inter-agent coordination success rate
- System availability and uptime
- Error recovery effectiveness

4.8 Ethical Considerations and Compliance

4.8.1 Responsible AI Principles

Transparency

- Clear disclosure of LLM involvement in decision processes
- Explainable outputs for all automated recommendations
- Human oversight for high-stakes decisions

Fairness

- Continuous bias monitoring
- Equitable treatment across customers and vendors
- Periodic fairness audits

Accountability

- Clearly defined responsibility chains
- Appeal and manual review mechanisms
- Immutable audit logs

Privacy

- Data minimization by design
- Secure handling of sensitive information
- Regulatory compliance enforcement

5. Expected Outcomes

5.1 Primary Deliverables

5.1.1 Functional FAgentLLM Prototype

The primary outcome of this project is the development of FAgentLLM, a functional prototype of a multi-agent, LLM-orchestrated financial automation system designed to enhance enterprise financial intelligence.

The system will consist of five specialized agents:

1. **Invoice Management Agent**

Responsible for extracting structured information from heterogeneous invoice formats, supporting automated approval workflows, and enabling integration with e-invoicing systems. The agent targets an extraction accuracy of approximately **85%** under controlled evaluation conditions.

2. **Budget Management Agent**

Provides centralized budget monitoring, variance tracking, and short-term forecasting to support proactive financial planning and decision-making.

3. **Reconciliation Agent**

Automates transaction matching across financial records, identifies discrepancies, and maintains traceable audit logs to support transparency and compliance.

4. **Credit Tracking and Collection Agent**

Supports credit risk assessment and structured collection workflows, enabling improved receivables monitoring and prioritization.

5. **Cash Management Agent**

Consolidates account balances and produces short-horizon cash flow forecasts to assist liquidity management and operational decision support. These agents will coordinate through an event-driven, shared financial state, enabling cross-domain reasoning and consistent system behavior.

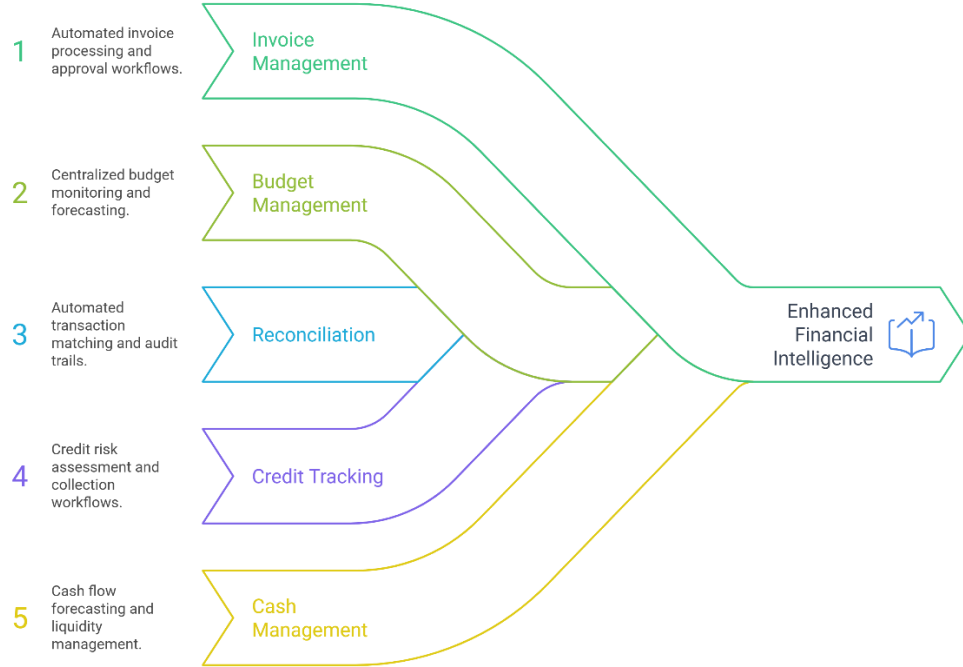


Figure 15 FAgentLLM's Financial Automation

5.1.2 System Integration and Operational Characteristics

The FAgentLLM prototype is expected to demonstrate:

- Inter-agent communication through defined orchestration protocols
- Integration capability with representative external systems (e.g., ERP-like data sources, banking records, invoice datasets)
- Modular and scalable architecture suitable for enterprise-level workflows
- Secure handling of sensitive financial data with logging and auditability mechanisms

The system will be implemented as a research-oriented prototype, focusing on architectural validity rather than full production deployment.

5.1.3 Technical and Research Documentation

The project will produce comprehensive documentation, including:

- System architecture diagrams and agent interaction models
- Description of data models, workflows, and orchestration logic
- Methodological justification for design choices and LLM integration
- Experimental setup, evaluation metrics, and performance analysis

This documentation will support both academic assessment and future system extension.

5.2 Research Contributions

5.2.1 Architectural Contribution

This project proposes a multi-agent LLM architecture in which Large Language Models function as cognitive reasoning components embedded within specialized financial agents, later will be as a centralized monolithic system.

Key contributions include:

- Modular intelligence through domain-specific agents
- Shared financial state enabling causal reasoning across financial processes
- Explainable automation via natural-language reasoning traces
- Adaptive handling of unstructured and evolving financial data

This architectural approach contributes to ongoing research on scalable and explainable enterprise AI systems.

5.2.2 Methodological Contribution

The project establishes practical design patterns for deploying LLMs in regulated financial environments, including:

- Hybrid architectures combining probabilistic LLM reasoning with deterministic validation
- Prompt engineering strategies for financial document understanding and decision support
- Reliability mechanisms such as validation rules and human-in-the-loop escalation
- Governance integration through logging, approvals, and compliance checks

These methods provide a structured framework for integrating LLMs into high-stakes business systems.

5.3 Practical Impact

5.3.1 Operational Benefits

The expected practical benefits of the proposed system include:

- Significant reduction in manual financial processing effort
- Improved data accuracy and consistency across financial operations
- Faster processing cycles for invoicing, reconciliation, and collections
- Enhanced decision support through consolidated, real-time financial visibility

While quantitative gains will be evaluated in a controlled environment, the results are expected to demonstrate clear efficiency and reliability improvements over traditional rule-based automation.

5.3.2 Applicability and Adoption Potential

FAgentLLM is designed to be adaptable across organizational contexts, including:

- Small and medium enterprises seeking cost-effective financial automation
- Large organizations requiring coordinated multi-department financial intelligence
- Public or regulated entities benefiting from transparency and auditability

The architecture supports flexible deployment models (on-premises, cloud, or hybrid), enabling broader applicability.

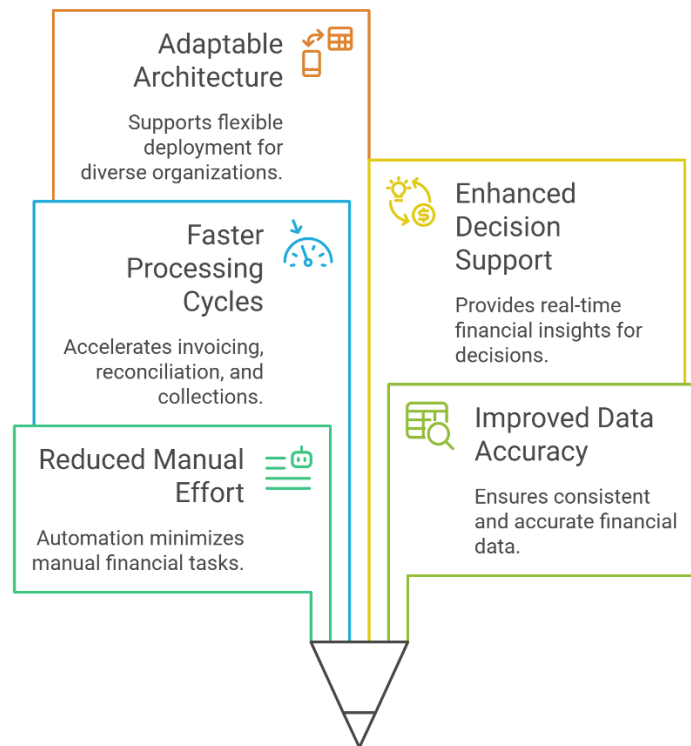


Figure 16 FAgentLLM's Financial Impact

5.4 Limitations and Future Work

5.4.1 Project Limitations

The project acknowledges the following limitations:

- Dependence on LLMs introduces computational overhead and probabilistic behavior
- Evaluation is conducted using synthetic or controlled datasets rather than full production data
- The scope is limited to five core financial domains
- The prototype emphasizes architectural validation rather than enterprise-scale optimization

5.4.2 Future Extensions

Future research may extend this work by:

- Incorporating reinforcement learning for adaptive decision optimization
- Expanding the agent framework to additional financial domains (e.g., tax, treasury, FP&A)
- Evaluating deployment in real organizational environments
- Exploring privacy-preserving multi-organization financial intelligence

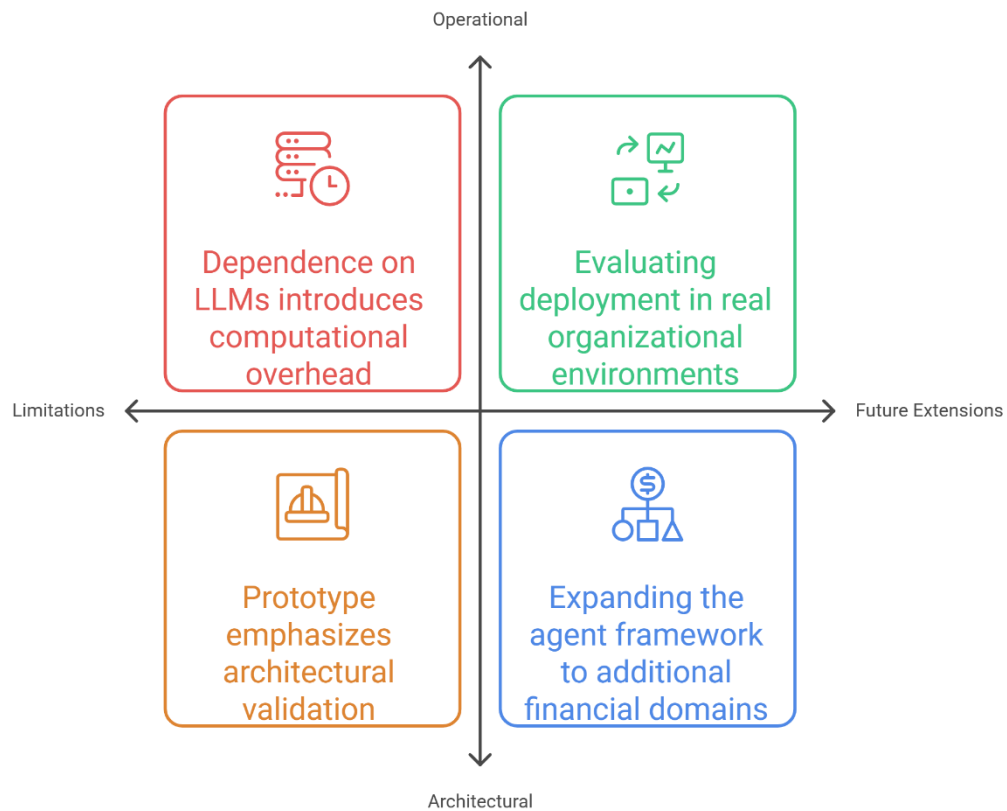


Figure 17 Project Limitations and Future Extensions

6. Timeline

The FAgentLLM project timeline spans January to May 2026, covering the transition from Capstone Project 1 (CP1) to Capstone Project 2 (CP2). Although Semester 7 officially concludes on 18 January 2026, a mid-semester academic break extends until 16 February 2026. This period is strategically used for individual technical deepening and preparatory study, allowing the team to enter Semester 8 with a high level of readiness and to accelerate core implementation activities.

The overall timeline is organized into three sequential phases, ensuring a logical progression from preparation to execution and final delivery:

1. Preparation and Knowledge Deepening (January – Mid February)
2. Core Implementation and Evaluation (Mid February – April)
3. Reporting, Discussion, and Final Presentation (April – May)

In total, *twelve* core technical and documentation tasks have been defined. These tasks will be executed during the upcoming semester, supported by an initial preparation phase conducted during the academic break to reduce implementation risk and improve execution efficiency.

6.1 High-Level Project Phases (January–May 2026)

Phase	Task / Activity	Responsible	Period
<i>Phase 0</i>	Individual technical deepening and tool familiarization	All team members	Jan 18 – Feb 16
<i>Phase 1</i>	Final methodology refinement and system design freeze	All team members	Feb 16 – Mar 2
<i>Phase 2</i>	Agent-level implementation and optimization	Individual members	Mar 2 – Mar 23
<i>Phase 3</i>	System integration and orchestration testing	All members	Mar 23 – Apr 6
<i>Phase 4</i>	Results generation and evaluation	All members	Apr 6 – Apr 20
<i>Phase 5</i>	Final report writing and presentation preparation	All members	Apr 20 – May 10

The subsequent section presents a detailed execution-level breakdown of tasks aligned with these phases.

6.2 Detailed Execution Timeline (Capstone Project 2)

Note: Phase 0 is conducted during the academic break to ensure accelerated implementation once Semester 8 begins.

#	Task / Activity	Responsible Member	Period (Weeks)
1	Final model optimization and LLM hyperparameter tuning	Member 3 (Orchestration)	Weeks 1–2
2	OCR and multimodal extraction pipeline refinement	Member 1 (Operational)	Weeks 1–3
3	Budgeting and cash flow forecasting model implementation	Member 2 (Planning)	Weeks 2–4
4	Deliverable 1 – Final Methodology submission	All members	Week 4
5	Multi-agent system integration and orchestration testing	Member 3 & SWE Team	Weeks 5–6
6	Quantitative results generation (accuracy, F1, MAPE)	Members 1 & 2	Weeks 6–7
7	Deliverable 2 – Implementation & Results submission	All members	Week 8
8	Explainability and reasoning trace evaluation	Member 3	Week 9
9	Deliverable 3 – Discussion submission	All members	Week 10
10	Deliverable 4 – Conclusion submission	All members	Week 12
11	Final report compilation (CP1 + CP2 integration)	All members	Week 13
12	Deliverable 5 – Final presentation and jury evaluation	All members	Week 14

3.1.1 Project Timeline and Gantt Chart

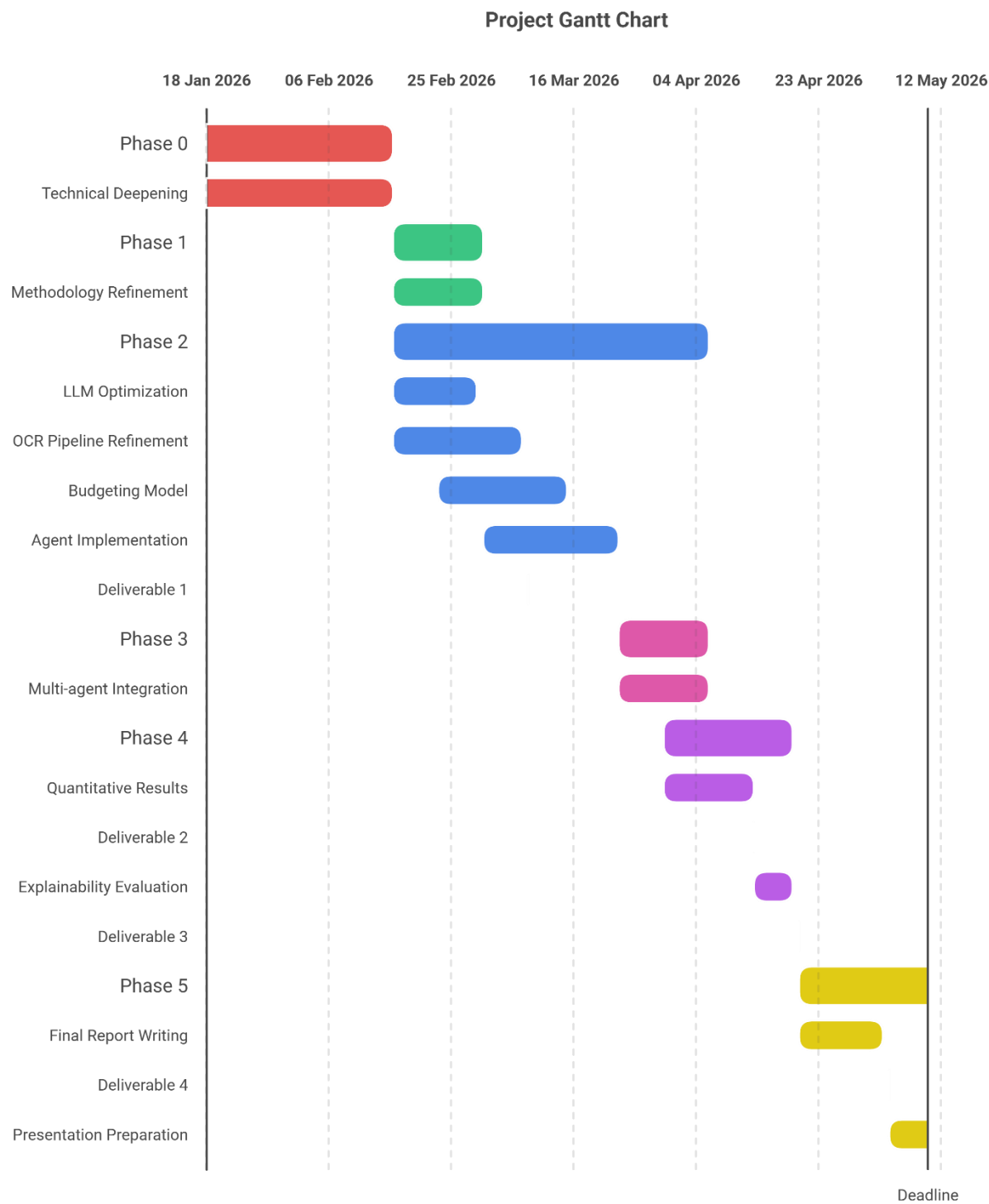


Figure 18 Gantt Chart of the FAgentLLM Project Timeline (January–May 2026)

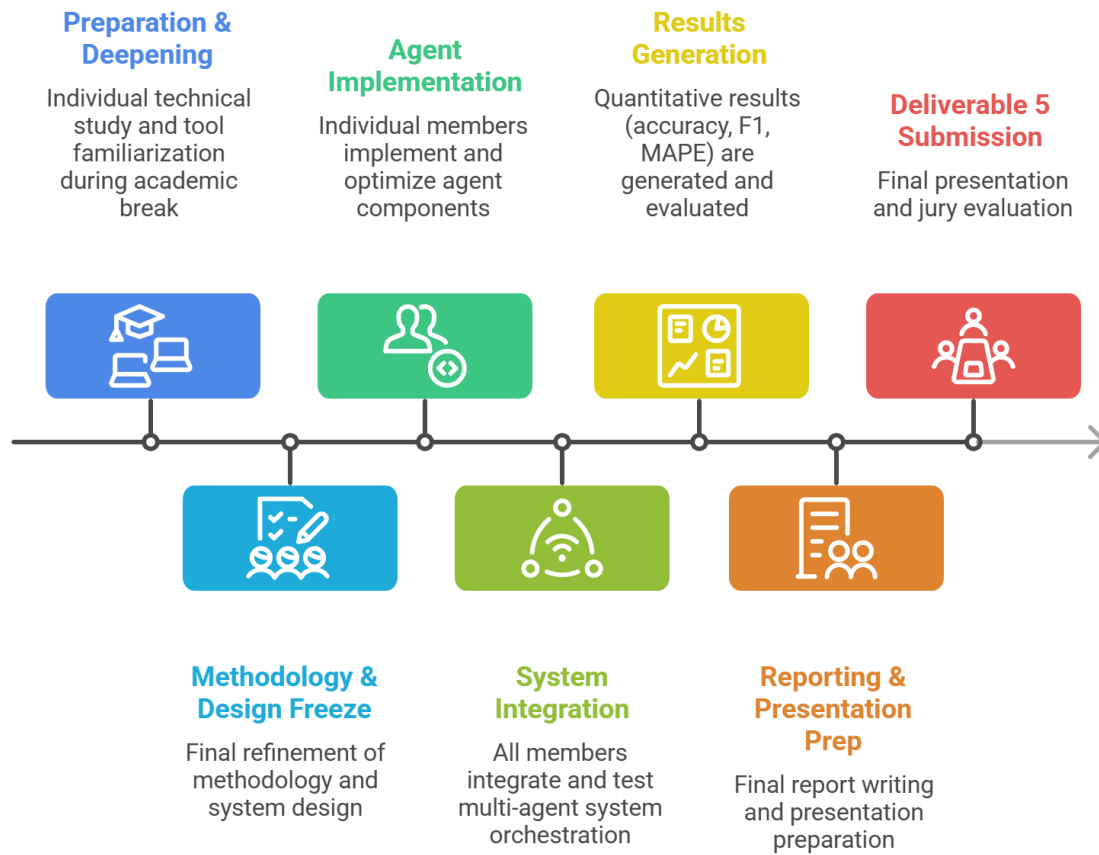


Figure 19 FAgentLLM Project Timeline (January–May 2026)

APPENDIX

Appendix A: Agent Interaction Scenarios

This appendix illustrates representative interaction scenarios within the **FAgentLLM multi-agent architecture**, demonstrating coordinated decision-making, inter-agent communication, and financial control enforcement across operational, planning, and orchestration layers.

Scenario 1: Invoice Approval with Liquidity-Aware Decision Support

This scenario demonstrates how invoice approval decisions are dynamically informed by real-time and forecasted cash positions.

1. The **Invoice Management Agent** receives a new vendor invoice with a total value of **\$50,000**.
2. The agent validates invoice completeness and compliance, determining that managerial approval is required based on predefined approval thresholds.
3. The **Invoice Management Agent** queries the **Cash Management Agent** to assess current liquidity and short-term cash flow projections.
4. The **Cash Management Agent** identifies a projected cash shortfall during the anticipated payment window.
5. Based on the liquidity assessment, the **Invoice Management Agent** flags the invoice for **finance manager review**, explicitly highlighting the identified liquidity risk.
6. Upon managerial approval, the **Invoice Management Agent** communicates the confirmed payment commitment to the **Cash Management Agent**.
7. The **Cash Management Agent** updates the cash flow forecast to reflect the newly committed liability.
8. The **Budget Management Agent** records the approved expenditure and updates departmental budget commitments accordingly.

Scenario 2: Reconciliation Discrepancy Triggering Credit Risk Reassessment

This scenario illustrates how payment behavior anomalies propagate through the system to inform credit risk management.

1. The **Reconciliation Agent** detects recurring payment delays associated with **Customer X**, exceeding normal variance thresholds.
2. The agent emits a **payment pattern anomaly event** to the shared event bus.
3. The **Credit Tracking Agent** subscribes to the event and retrieves the customer's historical payment data.
4. Using updated behavioral indicators, the **Credit Tracking Agent** recalculates the customer's credit risk score, resulting in an increased risk classification.
5. The agent updates the customer's credit limit and triggers escalation within the collections workflow.
6. The **Cash Management Agent** adjusts accounts receivable forecasts to reflect the revised payment risk for Customer X.

Scenario 3: Budget Threshold Breach Influencing Approval Workflows

This scenario highlights how budget utilization constraints dynamically alter invoice approval policies.

1. The **Budget Management Agent** detects that a department has exceeded **95% of its allocated quarterly budget**.
2. The agent broadcasts a **budget threshold breach event** to relevant system components.
3. The **Invoice Management Agent** receives the event and updates approval rules for the affected department.
4. Subsequent invoices associated with the department are automatically escalated to **senior management approval**.
5. The **Budget Management Agent** generates a formal alert notification to the department manager, outlining the budget status.
6. The **Cash Management Agent** reflects anticipated spending controls by revising the department's short-term expenditure forecast.

These scenarios collectively demonstrate the **adaptive, event-driven coordination** of FAgentLLM agents, enabling financially informed approvals, proactive risk management, and policy-driven governance. The interactions emphasize transparency, explainability, and real-time alignment between operational actions and financial constraints.

REFERENCES

- Chen, X., Li, Y., Cai, H., Ma, Z., Chen, X., Xiong, H., Wang, S., He, B., Sun, L., & Yin, D. (2025c). Multi-Agent proactive information seeking with adaptive LLM orchestration for Non-Factoid question answering. *KDD '25: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.*, 4341–4352. <https://doi.org/10.1145/3711896.3737249>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017c, June 12). *Attention is all you need*. arXiv.org. <https://arxiv.org/abs/1706.03762>
- LLM-based Multi-Agent Systems: Techniques and Business Perspectives*. (n.d.). <https://arxiv.org/html/2411.14033>
- Alonso-Rocha, J., Martínez-Rojas, A., Enríquez, J., & Sánchez-Oliva, J. (2025c). From manual to automated: a state-of-the-art review to examine the impact of intelligent document processing in banking automation. *Expert Systems With Applications*, 285, 127958. <https://doi.org/10.1016/j.eswa.2025.127958>
- Krieger, F., Drews, P., & Funk, B. (2023b). Automated invoice processing: Machine learning-based information extraction for long tail suppliers. *Intelligent Systems With Applications*, 20, 200285. <https://doi.org/10.1016/j.iswa.2023.200285>
- LLM-centric pipeline for information extraction from invoices*. (2024b, November 26). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/10852504>
- Towards a conversational invoice issuance LLM-Based agent*. (2024b, December 20). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/10899737>
- Huang, S. (2024b). Design and implementation of an intelligent financial management system based on enterprise legal system. *Journal of Organizational and End User Computing*, 36(1), 1–25. <https://doi.org/10.4018/joeuc.350224>
- Martseniuk, Y., Partyka, A., Opirskyy, I., & Harasymchuk, O. (2026). *Cost observability as a security control in multi-cloud environments based on SOC 2 security standard*. <https://www.semanticscholar.org/paper/Cost-observability-as-a-security-control-in-based-2-Martseniuk-Partyka/c5841d317becdb3faa93e63502ed59c23a4a9df9>
- Long Short-Term Memory (LSTM) networks for stock market prediction*. (2023, December 12). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/10449613>
- Enhancing Cash Flow Management with ML Approaches and Cloud-Based Tools*. (2025b, August 22). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/abstract/document/11199178>
- Tan, N., Vrochidis, I., Luscombe, H., Richardson, E., Plazas-Niño, F., Alexander, K., Martindale, L., Fields, N., Howells, M., Foster, V., & Harrison, J. (2025). IMPACCT: An integrated assessment model for policy and financial decision-making in energy planning. *Energy Strategy Reviews*, 62, 101981. <https://doi.org/10.1016/j.esr.2025.101981>

- Bakumenko, A., & Elragal, A. (2022b). Detecting anomalies in financial data using machine learning algorithms. *Systems*, 10(5), 130. <https://doi.org/10.3390/systems10050130>
- Advancing anomaly detection: Non-Semantic financial data encoding with large language models*. (2025). IEEE Journals & Magazine | IEEE Xplore. <https://ieeexplore.ieee.org/document/11131120>
- Scalable invoice reconciliation for SMEs: Edge-Deployed LLMs in Multi-Agent systems*. (2025b, March 28). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/abstract/document/11011581>
- Deep learning based anomaly detection for realtime financial transactions in cloud environments*. (2025b, August 5). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/11299653>
- Khalil, M. A., Hadid, M., Padmanabhan, R., Elomri, A., & Kerbache, L. (2025b). An integrated Artificial Intelligence and optimization model for operational efficiency and risk reduction in Letter of Credit examination process. *Decision Analytics Journal*, 14, 100552. <https://doi.org/10.1016/j.dajour.2025.100552>
- Peng, Q., Zhu, Y., & Wang, G. (2025b). SCF credit risk assessment with limited labeled data using label propagation algorithm and complex network approaches. *International Review of Financial Analysis*, 107, 104619. <https://doi.org/10.1016/j.irfa.2025.104619>
- Fatemi, S., & Hu, Y. (2024b). FinVision: a Multi-Agent Framework for Stock market Prediction. *ICAIF '24: Proceedings of the 5th ACM International Conference on AI in Finance*, 582–590. <https://doi.org/10.1145/3677052.3698688>
- Zhao, H., Liu, Z., Wu, Z., Li, Y., Yang, T., Shu, P., Xu, S., Dai, H., Zhao, L., Jiang, H., Pan, Y., Chen, J., Zhou, Y., Zhang, Z., Sun, R., Mai, G., Liu, N., & Liu, T. (2024b, January 22). *Revolutionizing Finance with LLMs: An Overview of Applications and Insights*. arXiv.org. <https://arxiv.org/abs/2401.11641>